

Instruções gerais

1. Esta avaliação pode ser executada em grupo de até dois integrantes e tem por objetivo consolidar o aprendizado sobre o projeto RTL usando VHDL.
2. Os circuitos devem ser descritos em VHDL e verificados com o uso de testbenches no simulador ModelSim.
3. Os circuitos implementados devem ser sintetizados no Quartus Prime e verificados utilizando o visualizador de RTL para conferir os circuitos inferidos.
4. Para cada implementação, deve-se utilizar o Quartus Prime para identificar os custos de implementação do circuito expresso pelo número de LCs, LUTs e Registradores (*i.e.* Flip-Flops – FFs) ocupados (conforme a família de dispositivos utilizada), e a Frequência de operação máxima (Fmax) obtida.
5. Aplique o seguinte guia de estilo na codificação VHDL:
 - a. Tabulação com 02 espaços (substituindo tabs por espaços);
 - b. Palavras reservadas da linguagem em caixa baixa;
 - c. Nomes de sinais em caixa alta (exceto os prefixos);
 - d. Utilize os seguintes prefixos para identificar o tipo sinal:
 - i_NOME : pino de entrada
 - o_NOME : pino de saída
 - b_NOME : pino de entrada-e-saída (bidirecional)
 - w_NOME : fio interno
 - v_NOME : variável de processo
 - r_NOME : registrador
 - t_NOME : tipo de dado
 - p_NOME : processo
 - u_NOME : instância de componente
 - s_NOME : estado de uma FSM
 - e. Utilize o sufixo “n” para sinalizar que o sinal é ativo em 0 (negado). Ex: i_RSTn
6. Deve ser entregue um relatório em formato PDF que contenha:
 - a. Identificação do autor e do trabalho (Avaliação 3)
 - b. Enunciado de cada circuito e, para cada um deles:
 - Códigos em linguagem C da função solicitada¹;
 - Códigos VHDL do circuito e do testbench;
 - Diagrama de formas de onda da simulação, marcando os pontos de interesse que evidenciam o correto funcionamento do circuito;
 - Diagrama RTL gerado pelo Quartus mostrando a inferência do circuito implementado;
 - Tabela de custos e desempenho do circuito, incluindo as quantidades de células lógicas, LUTs e Flip-Flops ocupados, além da Fmax;
 - Textos de análise dos resultados obtidos, *i.e.* simulação, diagrama RTL e recursos ocupados.

¹ Para apresentar os códigos, utilize fonte monoespaçada, tal como a Courier New, com tamanho 10.

7. O relatório deve ser entregue em formato PDF, postado no ambiente AVA, até a data limite de entrega (12/11).
8. Os grupos deverão implementar uma dentre duas alternativas de sistema, sendo essas denominadas **Sistemas Básico** e **Sistema Avançado**.

Sistema Básico – MDC

Dado o algoritmo do máximo divisor comum (MDC) descrito a seguir:

```
1:  int x, y;                      // variáveis internas
2:  while (1) {
3:      while (!i_go);              // i_go : entrada de 1 bit
4:      o_rdy = 0;                  // o_rdy: saída de 1 bit
5:      x = i_x;                    // i_x  : entrada de 8 bits
6:      y = i_y;                    // i_y  : entrada de 8 bits
7:      while (x != y) {
8:          if (x < y)
9:              y = y - x;
10:         else
11:             x = x - y;
12:     }
13:     o_d = x;                     // o_d: saída de 8 bits
14:     o_rdy = 1;
15: }
```

Projete e implemente um sistema digital no nível RT que implemente esse algoritmo.

Requisitos:

1. O sistema deve ser composto de um controle (máquina de estados) e um caminho de dados;
2. O projeto deve seguir uma abordagem hierárquica, com a seguinte estrutura sugerida
 - a. Topo (MDC – ex. top.vhd)
 - b. Controle (máquina de estados – ex. fsm.vhd ou control.vhd)
 - c. Caminho de dados (ex. datapath.vhd)
 - Interconexão de blocos RT
3. Considere que as entradas de dado (*i_x* e *i_y*), a saída de dado (*o_d*), a saída de estado (*o_rdy*) e as variáveis internas (*x* e *y*) são números inteiros sem sinal de 8 bits;
4. Utilize o método de projeto RTL visto em sala de aula, o qual parte de um algoritmo C e a partir dele obtém uma máquina de estados de alto nível e desta extrai os blocos de controle (fsm) e de processamento (caminho de dados);
5. Ao realizar o projeto, identifique previamente e claramente os nomes das entradas, saídas e sinais internos nos diagramas esquemáticos de modo a facilitar a documentação e a descrição do projeto em VHDL; e
6. Para sinais internos, utilize como referência a porta de saída e o nome do bloco ao qual o sinal está conectado. Exemplo: para um sinal de 8 bits conectado à saída de um somador, utilize o seguinte identificador: *w_ADDER_OUT*. Caso o sistema contenha mais de uma instância de um mesmo bloco, diferencie-os com algum sufixo (ex. *w_ADDER0_out*, *w_ADDER1_out*,...). Nota: os exemplos utilizados são apenas a título de ilustração, pois o algoritmo não utiliza operação de soma.

Sistema Avançado

Selecione um algoritmo de alto nível descrito em linguagem C e aplique o método de projeto RTL para projetar e implementar um processador de propósito específico para execução do algoritmo diretamente em hardware.

O algoritmo selecionado deve ser informado aos professores, receber a confirmação de aderência ao escopo da atividade, e depois comunicado aos demais alunos no grupo de discussão para que o mesmo algoritmo não seja selecionado por mais de um grupo.

Requisitos

1. A função deve ser implementada em linguagem C e utilizada como referência para o sistema digital;
2. O sistema digital deve ser composto de um controle e um caminho de dados;
7. O projeto deve seguir uma abordagem hierárquica, com a seguinte estrutura sugerida:
 - a. control.vhd (máquina de estados);
 - b. datapath.vhd (caminho de dados);
 - c. top.vhd (integração da máquina de estados e do caminho de dados);
8. Considere as entradas e saídas de dados representados como std_logic_vector;
9. Utilize o método de projeto RTL visto em sala de aula, o qual parte de um algoritmo C e a partir dele obtém uma máquina de estados de alto nível e desta extrai os blocos de controle e de processamento (caminho de dados);
10. Ao realizar o projeto, identifique claramente os nomes das entradas, saídas e sinais internos nos diagramas esquemáticos de modo a facilitar a documentação e a descrição do projeto em VHDL;
11. Inclua um sinal de entrada para indicar quando o processamento deve ser iniciado (ex. i_GO ou i_START);
12. Inclua um sinal de saída para indicar quando o processamento for concluído (ex. o_RDY). Este sinal deve ser igual a 0 enquanto o processamento estiver em andamento;
13. Para o projeto, procure utilizar alguma ferramenta de apoio (sugestão draw.io) para documentar e facilitar a implementação posterior.